Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences

FACHBEREICH
INFORMATIK

Programmierung 1

Primitive Datentypen

WS 2026/27

Bachelor Cyber Security & Privacy (B.Sc.)

André Kless, M.Sc.

Inhalte dieser Vorlesung

Wahrheitswerte

`boolean`: Bedingungen, logische Verknüpfungen und Kurzschlussauswertung

Ganzzahlen

Stellenwertsysteme, Zweierkomplement, Rechnen und einzelne Bits

Gleitkommazahlen

`float` und `double`: Kodierung, Genauigkeit und Typumwandlungen

Zeichen und Zeichenketten

ASCII, Unicode, `char` und ergänzend `String`

Was legt ein Datentyp fest?

```
int anzahl = 3;
boolean istBereit = true;
```

Jede Variable und jeder Ausdruck hat einen Datentyp.
Er bestimmt die möglichen Werte und die erlaubten Operationen.

Gruppe	Datentypen	Beispiele für Werte
Wahrheitswerte	`boolean`	`true`, `false`
Ganze Zahlen	`byte`, `short`, `int`, `long`	`-7`, `0`, `42`
Gleitkommazahlen	`float`, `double`	`0.5f`, `3.14`
Zeichen	`char`	`'A'`, `'7'`, `'\n'`
Zeichenketten (Zusatz)	`String`	`"Hallo"`, `""`

Java 25: Datentypen, Werte und Variablen

Ein Bit: zwei mögliche Zustände

Ein Bit ist die kleinste Informationseinheit.
Es kann einen von zwei Werten annehmen: `0` oder `1`.

Beispiele für mögliche Kodierungen:

Bitwert	Lampe	Tür	Antwort
`0`	aus	geschlossen	nein
`1`	an	offen	ja

Mit einem Bit lassen sich zwei Möglichkeiten unterscheiden.

Für mehr Möglichkeiten verwenden wir mehrere Bits.

Mit drei Bits acht Zustände kodieren

Mit jedem zusätzlichen Bit verdoppelt sich die Anzahl möglicher Zustände.
Beispiel: Wir ordnen diesen Zuständen ganze Zahlen ab 0 zu.

3. Bit	2. Bit	1. Bit	Kodierte Zahl
0	0	0	**0**
0	0	1	**1**
0	1	0	**2**
0	1	1	**3**
1	0	0	**4**
1	0	1	**5**
1	1	0	**6**
1	1	1	**7**

1 Bit: $2^1 = 2$ Zustände

also 2 Zahlen: 0 und 1

2 Bits: $2^2 = 4$ Zustände

also 4 Zahlen: 0 bis 3

3 Bits: $2^3 = 8$ Zustände

also 8 Zahlen: 0 bis 7

Die Tabelle zeigt die Zuordnung:
Jeder Zustand steht für eine Zahl.

Mit 0 und 1 die Stellenwerte auswählen

Von rechts nach links sind die Stellenwerte $2^0 = 1$, $2^1 = 2$ und $2^2 = 4$.
Mit `1` zählt der Stellenwert mit. Mit `0` lassen wir ihn weg.

$2^2 = 4$	$2^1 = 2$	$2^0 = 1$	Kodierte Zahl	Berechnung des Zahlenwerts
0	0	0	0	$2^2 \cdot 0 + 2^1 \cdot 0 + 2^0 \cdot 0 = 0$
0	0	1	1	$2^2 \cdot 0 + 2^1 \cdot 0 + 2^0 \cdot 1 = 1$
0	1	0	2	$2^2 \cdot 0 + 2^1 \cdot 1 + 2^0 \cdot 0 = 2$
0	1	1	3	$2^2 \cdot 0 + 2^1 \cdot 1 + 2^0 \cdot 1 = 3$
1	0	0	4	$2^2 \cdot 1 + 2^1 \cdot 0 + 2^0 \cdot 0 = 4$
1	**0**	**1**	**5**	$\mathbf{2^2 \cdot 1 + 2^1 \cdot 0 + 2^0 \cdot 1 = 5}$
1	1	0	6	$2^2 \cdot 1 + 2^1 \cdot 1 + 2^0 \cdot 0 = 6$
1	1	1	7	$2^2 \cdot 1 + 2^1 \cdot 1 + 2^0 \cdot 1 = 7$

Ein Byte: acht Bits und 256 Zustände

Ein Byte besteht aus acht Bits.
Jedes Bit kann unabhängig von den anderen `0` oder `1` sein.

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
0	0	1	0	1	0	1	0

`00101010` kodiert: $2^5 + 2^3 + 2^1 = 32 + 8 + 2 = 42$

Größter Wert: alle acht Bits sind 1

$2^7 \cdot 1 + 2^6 \cdot 1 + 2^5 \cdot 1 + 2^4 \cdot 1 + 2^3 \cdot 1 + 2^2 \cdot 1 + 2^1 \cdot 1 + 2^0 \cdot 1$

$= 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$

Zahlenbereich ohne Vorzeichen: 0 bis 255

Mit der Null sind das $255 + 1 = 256 = 2^8$ verschiedene Werte.

Clonk Rage: Eigenschaften als Bitmaske

Beispiel aus meiner Kindheit

`Category` kodiert mehrere Eigenschaften eines Spielobjekts.
Jedes Bit steht für eine Eigenschaft: `1` = ja, `0` = nein.



Beispiel Armbrust: `Category=3332`

2^{11}	2^{10}	2^{9}	2^{8}	2^{7}	2^{6}	2^{5}	2^{4}	2^{3}	2^{2}	2^{1}	2^{0}
1	**1**	0	**1**	0	0	0	0	0	**1**	0	0

$2^{11} + 2^{10} + 2^{8} + 2^{2} = 2048 + 1024 + 256 + 4 = 3332$

20 führende Nullen weggelassen.

$2^{11} = 2048$	`C4D_SelectHomebase`	Als Angebot der Heimatbasis auswählbar
$2^{10} = 1024$	`C4D_SelectKnowledge`	Als Bauplan auswählbar
$2^{8} = 256$	`C4D_SelectVehicle`	Als Startfahrzeug auswählbar
$2^{2} = 4$	`C4D_Vehicle`	Fahrzeug

Quellen: Armbrust-Definition · Objektkategorien

Spielgrafik: RedWolf Design (CC BY-NC 4.0) · Screenshot: A. Kless, Ausschnitt

Speichergrößen: von Byte bis Petabyte

Diese Angaben kennen wir von Dateigrößen, USB-Sticks und Festplatten.

Diese Menge	Entspricht	Name
8 Bit	1 B	Byte
1.000 B	1 kB	Kilobyte
1.000 kB	1 MB	Megabyte
1.000 MB	1 GB	Gigabyte
1.000 GB	1 TB	Terabyte
1.000 TB	1 PB	Petabyte

Beispiel: 1-TB-Festplatte = 1.000 GB = 1.000.000.000.000 Byte

Bei kB, MB, GB, TB und PB gelten 1.000er-Schritte.

Quelle: NIST – Einheiten und Präfixe

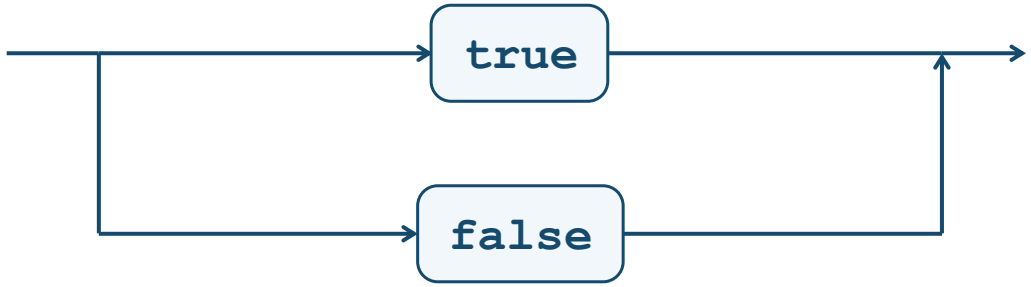
Wahrheitswerte

`boolean`



Wahrheitswerte mit `boolean`

<Wahrheitswertliteral>




```
boolean istBereit = true;
boolean istFertig = false;
boolean hatRest = 7 % 2 != 0;
```

Zur Unterscheidung von `true` und `false` genügt ein Bit.

Die JVM kann größere Speichereinheiten nutzen. Wie viel Speicher ein `boolean` belegt, hängt von der JVM und der Verwendung ab.

Beispiel mit einem Byte: `false` als `00000000`, `true` als `00000001`.

`true` und `false` sind keine Zahlen. `boolean b = 1;` ist unzulässig.

JVM-Spezifikation: Darstellung von boolean

Logische Operatoren und Wahrheitstabellen

a	b	!a	a && b	a || b	a ^ b
false	false	true	false	false	false
false	true	true	false	true	true
true	false	false	false	true	true
true	true	false	true	true	false

`!` kehrt den Wahrheitswert um. `&&` ist nur bei zweimal `true` wahr.
`||` ist bei mindestens einmal `true` wahr.
`^` ist genau dann wahr, wenn die beiden Werte verschieden sind.

Java 25: logische Operatoren und Auswertung

Kurzschlussauswertung: `&&` und `||`

```
int teiler = 0;
boolean passt = teiler != 0 && 12 % teiler == 0;
```

Operator	Wann wird der rechte Ausdruck ausgewertet?
`a && b`	Nur wenn `a` den Wert `true` hat.
`a || b`	Nur wenn `a` den Wert `false` hat.
`a & b`, `a | b`, `a ^ b`	Immer, sofern die linke Auswertung regulär endet.

Im Beispiel ist der linke Teil `false`. Die Division durch null wird vermieden.
Mit `&` würde Java auch den rechten Teil auswerten: Laufzeitfehler.

Weitere Ausdrücke mit Wahrheitswerten

Form	Beispiel	Bedeutung
Gleichheit / Ungleichheit	`a == b`, `a != b`	Wahrheitswerte vergleichen
Kurzzuweisungen	`a &= b; a |= b;` `a ^= b;`	Verknüpfen und in `a` speichern
Klammerung	`!(a || b)` `!a && !b`	Beide Ausdrücke sind gleichwertig.

Bedingter Ausdruck: einen von zwei Werten auswählen



`int rabatt = istMitglied ? 10 : 0;` Bei `true`: `10`, sonst `0`.

Ganzzahlen

`byte, short, int, long`



Stellenwertsysteme

Die Basis legt den Ziffernvorrat und die Gewichte der Stellen fest.

System	Basis	Ziffern	Derselbe Wert
Dual	2	`0`, `1`	101010_2
Oktal	8	`0` bis `7`	52_8
Dezimal	10	`0` bis `9`	42_{10}
Hexadezimal	16	`0` bis `9`, `A` bis `F`	$2A_{16}$

Eine tiefgestellte Basis kennzeichnet hier das Zahlensystem.
Diese mathematische Schreibweise ist noch kein Java-Code.

Der Wert einer Ziffer hängt von ihrer Stelle ab

Dezimal: 347_{10}

Ziffer	3	4	7
Stellenwert	$10^2 = 100$	$10^1 = 10$	$10^0 = 1$
Beitrag	300	40	7

Dual: 101010_2

$1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 32 + 8 + 2 = 42$

Oktal: 52_8

$5 \cdot 8^1 + 2 \cdot 8^0 = 40 + 2 = 42$

Hexadezimal: $2A_{16}$ (A = 10)

$2 \cdot 16^1 + 10 \cdot 16^0 = 32 + 10 = 42$

Allgemein: Jede Ziffer wird mit Basis^Position multipliziert. Rechts beginnt Position 0.

Dezimal in dual, oktal und hexadezimal umrechnen

Wiederholt durch die Basis teilen. Den ganzzahligen Quotienten weiterverwenden.
Die Reste von unten nach oben lesen.

Dual: $42_{10} = 101010_2$

Division	Quotient	Rest
42 / 2	21	0
21 / 2	10	1
10 / 2	5	0
5 / 2	2	1
2 / 2	1	0
1 / 2	0	1

Aufhören, sobald der Quotient 0 ist.

Oktal: $42_{10} = 52_8$

Division	Quotient	Rest
42 / 8	5	2
5 / 8	0	5

Hexadezimal: $42_{10} = 2A_{16}$

Division	Quotient	Rest
42 / 16	2	10 = A
2 / 16	0	2

Oktal und hexadezimal über Bitgruppen

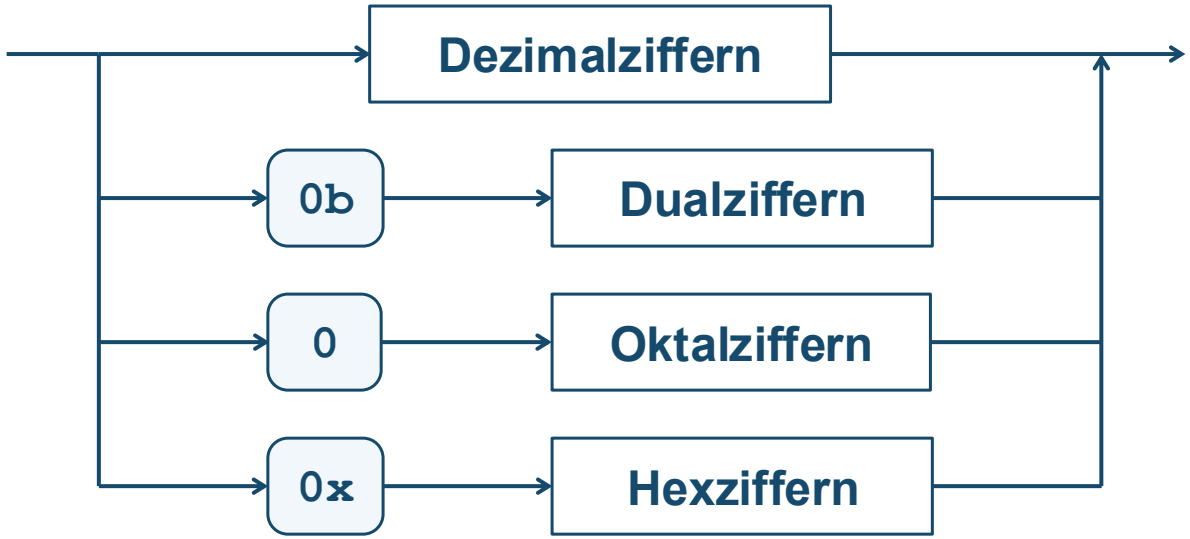
Darstellung	Gruppierung	Wert
Dual	`101010`	42_{10}
Oktal: 3 Bits je Ziffer	`101 010`	5 2 $\rightarrow$ 52_8
Hexadezimal: 4 Bits je Ziffer	`0010 1010`	2 A $\rightarrow$ $2A_{16}$

Von rechts gruppieren. Links darf man mit Nullen ergänzen.
Eine Hexadezimalziffer steht für vier Bits, zwei Ziffern für ein Byte.

A = 10, B = 11, C = 12, D = 13, E = 14, F = 15

Ganzzahlliterale in Java

<Ganzzahlliteral>




```
int a = 42;
int b = 0b101010;
int c = 052;
int d = 0x2A;
```

Alle vier Variablen enthalten denselben Wert: 42.

Das Diagramm zeigt die Grundformen. Dezimal beginnt nur die Null selbst mit `0`.
Auch `0B`, `0X` und kleine Hexadezimalbuchstaben sind erlaubt.

Vollständige Regeln: Ganzzahlliterale (JLS §3.10.1)

Wie negative Zahlen kodieren?

Erste Idee: Bei 8 Bits übernimmt das linke Bit das Vorzeichen.
`0` steht für Plus, `1` für Minus. Die übrigen 7 Bits kodieren den Betrag.

Vorzeichen (1 Bit)	Betrag (7 Bits)	Zahl
`0`	`0000101`	+5
`1`	`0000101`	−5
`0`	`0000000`	+0
`1`	`0000000`	−0

Der Betrag reicht nur noch von 0 bis 127 statt von 0 bis 255.

+0 und −0 sind dieselbe Zahl: 256 Bitkombinationen, aber 255 verschiedene Werte.

Ziel: alle 256 Bitkombinationen für unterschiedliche Werte nutzen.

Negative Ganzzahlen: Zweierkomplement

−128	64	32	16	8	4	2	1
1	0	0	0	0	1	0	1

Das höchste Bit hat beim Zweierkomplement ein negatives Gewicht.
Hier: −128 + 4 + 1 = −123.

Bei n Bits	Mit 8 Bits
Kleinster Wert: -2^{n-1}	−128: `10000000`
Größter Wert: $2^{n-1} - 1$	127: `01111111`

Führende Bits und Vorzeichenerweiterung

Bitbreite	Darstellung von −5	Berechnung
4 Bits	`1011`	−8 + 2 + 1 = −5
5 Bits	`11011`	−16 + 8 + 2 + 1 = −5
8 Bits	`11111011`	−128 + 64 + 32 + 16 + 8 + 2 + 1 = −5

Beim Erweitern gleichen sich die neuen Gewichte aus: −16 + 8 = −8.

Auffüllen: höchstes Bit wiederholen

Negative Zahlen mit `1`, positive und null mit `0`.
Beispiel +5: `0101` wird zu `00000101`.

Verkürzen: Vorzeichen erhalten

Links nur entfernen, wenn die beiden höchsten Bits gleich sind.
`1011` darf nicht zu `011` werden (+3).

Java: `byte` 8, `short` 16, `int` 32, `long` 64 Bits. Die Bitbreite ist fest.

Im Quelltext: `byte zahl = -5;  0b1011` bedeutet +11.

Java-Spezifikation: feste Bitbreiten und Vorzeichenerweiterung

Von positiven zu negativen Ganzzahlen

Beispiel mit 8 Bits: Wie kodieren wir −5?

Schritt	Bitmuster
Positive Zahl 5	`00000101`
Alle Bits umkehren	`11111010`
1 addieren: Darstellung von −5	`11111011`

Für negative Zahlen: positives Bitmuster umkehren und 1 addieren.
Die Addition 5 + (−5) ergibt `1 00000000`. In 8 Bits bleibt `00000000`.

`byte`: eine ganze Zahl mit 8 Bits

256 mögliche Werte im Zweierkomplement: −128 bis 127

```
byte temperatur = -5;
byte anzahl = 127;
// byte zuGross = 128; // Compilerfehler
```

Geeignet, wenn ein kleiner Wertebereich ausdrücklich genügt.
Ein Java-`byte` ist vorzeichenbehaftet. Es stellt nicht den Bereich 0 bis 255 dar.

Ein Byte als Speichereinheit beschreibt acht Bits. Der Java-Typ `byte` legt zusätzlich fest, wie diese Bits als Zahl interpretiert werden.

Java 25: Wertebereiche der ganzzahligen Datentypen

`short`: eine ganze Zahl mit 16 Bits

65.536 mögliche Werte im Zweierkomplement: −32.768 bis 32.767

```
short hoehe = 32000;
short tiefsterWert = -32768;
// short zuGross = 32768; // Compilerfehler
```

Zwei Bytes pro kodiertem Wert.
Sinnvoll bei passend begrenzten Daten, etwa einzelnen 16-Bit-Messwerten.

Auch `short` verwendet das Zweierkomplement. Die zusätzliche Breite vergrößert den Wertebereich, die Grundidee der Kodierung bleibt gleich.

`int`: der übliche Typ für ganze Zahlen

32 Bits im Zweierkomplement: −2.147.483.648 bis 2.147.483.647

Unterstriche zwischen Ziffern erleichtern das Lesen, z.B. in Tausendergruppen: `1_000_000`.

```
int anzahl = 1_000_000;
int jahr = 2028;
int groessterWert = 2_147_483_647;
```

Der Zahlenwert bleibt gleich: `1_000_000` entspricht `1000000`.

Für Zähler, Indizes und viele ganzzahlige Berechnungen reicht `int` aus.
Die Breite ist in Java festgelegt und hängt nicht vom Betriebssystem ab.

`3_000_000_000` passt nicht mehr in `int`.
Für solche Werte brauchen wir einen Ganzzahltyp mit größerem Wertebereich.

`long`: ganze Zahlen mit großem Wertebereich

64 Bits im Zweierkomplement

−9.223.372.036.854.775.808 bis 9.223.372.036.854.775.807

```
long anzahl = 3_000_000_000L;
long sekunden = 86_400L;
long kopie = 42;
```

Das `L` gehört zum Literal. Es sorgt dafür, dass dieses bereits den Typ `long` hat.
Der kleine Wert `42` passt zunächst in `int` und lässt sich automatisch erweitern.

Auch `long` hat Grenzen. Für beliebig große ganze Zahlen gibt es später weitere Werkzeuge.

64-Bit-Prozessoren und Java-Datentypen

64 Bit bei der Hardware

Register sind kleine, sehr schnelle Speicherplätze direkt im Prozessor.
Seine allgemeinen Register fassen 64 Bit, z. B. für das Rechnen mit Ganzzahlen.
Der Prozessor kann auch mit kleineren Bitbreiten arbeiten, z. B. mit 32 Bit.

Feste Bitbreiten in Java

Datentyp	`byte`	`short`	`int`	`long`
Bitbreite des Werts	8 Bit	16 Bit	32 Bit	64 Bit

Die Bitbreiten und Rechenregeln legt Java unabhängig vom Prozessor fest.
`int` bleibt 32 Bit. `long` ist auf einem 64-Bit-Prozessor nicht automatisch schneller.
Den Typ nach dem Wertebereich wählen. Kleinere Datentypen können Speicher sparen.

x64-Architektur (Microsoft) · Java 25: primitive Datentypen

Ganzzahlliterale: Typ und Schreibweise

Schreibweise	Beispiel	Bedeutung
Ohne Suffix	`42`	Typ `int`
Mit `L` (auch `l` erlaubt)	`3_000_000_000L`	Typ `long`, großes `L` ist lesbarer.
Unterstriche zwischen Ziffern	`1_000_000`, `0xFF_FF`	Nur zur Gliederung, ohne Einfluss auf den Wert.
Führende Null	`010`	Oktal: Wert 8. `08` ist kein gültiges Literal.

Das Minus ist ein Operator vor dem Literal: `-42`.
Binäre, oktale und hexadezimale Literale können auch negative Bitmuster beschreiben:
`0xFFFFFFFF` hat als `int` den Wert `-1`.

Ganzzahlen: Rechnen und Vergleichen

Operatoren	Beispiel	Ergebnis
`+, -, *`	`7 + 3, 7 - 3, 7 * 3`	`10, 4, 21`
`/, %`	`7 / 3, 7 % 3`	`2, 1`
Unäres `+` und `-`	`+7, -7`	`7, -7`
`<, <=, >, >=`	`7 >= 3`	`true`
`==, !=`	`7 == 3`	`false`

Zuweisung: `=`. Rechnen und zuweisen: `+=, -=, *=, /=, %=`.
Vergleiche liefern immer einen Wert vom Typ `boolean`.

Ganzzahldivision und Rest

Ausdruck	Wert	Begründung
`7 / 3`	`2`	Der Nachkommaanteil entfällt.
`-7 / 3`	`-2`	Abschneiden in Richtung null.
`7 % 3`	`1`	$7 = 2 \cdot 3 + 1$
`-7 % 3`	`-1`	$-7 = (-2) \cdot 3 + (-1)$
`7 % -3`	`1`	Ein Rest ungleich null hat das Vorzeichen links.

`%` ist in Java der Restoperator, oft „Modulo“ genannt.
Ist der rechte Wert null, führen `/` und `%` bei Ganzzahlen zu einem Laufzeitfehler.

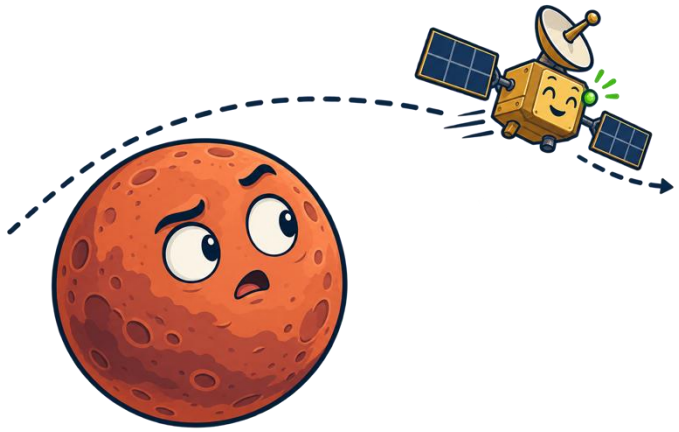
Überlauf bei Ganzzahlen

```
int maximum = 2_147_483_647;
IO.println(maximum + 1); // -2147483648
```

Das Ergebnis liegt außerhalb des `int`-Wertebereichs.
Java behält die unteren 32 Bits.

Keine automatische Fehlermeldung!

Das Programm rechnet mit dem falschen Wert weiter.



Dann trifft die Sonde den Mars nicht.

8-Bit-Modell	Bitmuster	Interpretation
127	`01111111`	größter positiver Wert
127 + 1	`10000000`	−128

Multiplikation: Überlauf bei `int`

```
int a = 100_000, b = 100_000;
IO.println(a * b); // 1410065408
```

Mathematisch: 100 000 · 100 000 = 10 000 000 000

fallen weg		**untere 32 Bits bleiben**
1	**0**	**0101 0100 0000 1011 1110 0100 0000 0000**
2^{33}	2^{32}	Höchstes verbleibendes Bit: 0. Das gespeicherte `int`-Ergebnis ist positiv.

Gespeichert: 1 410 065 408

Abweichung: 10 000 000 000 − 1 410 065 408
= 8 589 934 592 = $1 \cdot 2^{33} + 0 \cdot 2^{32}$

Das Ergebnis ist um 8 589 934 592 zu klein. Java meldet keinen Fehler.

Java 25: Multiplikation und Rundung

Inkrement und Dekrement als Ausdruck

Präfix: zuerst ändern

```
int a = 5;
int b = ++a;
// a: 6, b: 6
```

Postfix: zuerst alten Wert liefern

```
int a = 5;
int b = a++;
// a: 6, b: 5
```

Für `--a` und `a--` gilt das entsprechend mit einer Verringerung um 1.
Als eigene Anweisung bewirken `a++;` und `++a;` dieselbe Änderung.

Empfehlung: Änderungen möglichst als eigene Anweisung schreiben.
Mehrere Änderungen in einem Ausdruck sind schwer nachzuvollziehen.

Implizite Typumwandlung bei Ganzzahlen

Implizit bedeutet: Java führt die Umwandlung automatisch aus.
Beim Erweitern auf einen größeren Ganzzahltyp bleibt der Wert erhalten.




```
byte klein = 100;
int mittel = klein;
long gross = mittel;
```

Jeder Pfeil erlaubt auch den direkten Weg weiter nach rechts.
Beispiel: `byte` nach `long`.

Java 25: erweiternde primitive Typumwandlungen

Explizite Typumwandlung mit einem Cast

Explizit bedeutet: Wir geben den Zieltyp im Quelltext an.




```
int wert = 130;
byte klein = (byte) wert;
IO.println(klein); // -126
```

`130`: 00000000 … 10000010
Als `byte` bleiben 8 Bits.
`10000010` bedeutet dann −126.

Ein Cast kann Information verwerfen. Er garantiert nicht, dass der Wert erhalten bleibt.
„Implizites Casting“ meint meist eine implizite Umwandlung, ohne Cast im Quelltext.

Mit welchem Ganzzahltyp rechnet Java?

Für +, −, *, /, % und numerische Vergleiche

Mindestens ein Operand ist `long`: Beide Werte werden als `long` verarbeitet.
Sonst: Java verarbeitet die Werte als `int`, auch bei `byte` und `short`.

```
byte a = 10;
byte b = 20;
int summeAlsInt = a + b;
// byte summe = a + b; // Compilerfehler
byte summe = (byte) (a + b);
```

Der Cast wandelt das Ergebnis zurück in `byte`.
Er prüft dabei nicht, ob der Wert hineinpasst!

Als `long` rechnen: `long x = 50_000L * 50_000;`

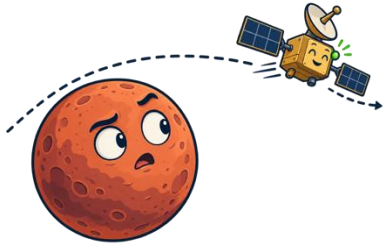
Der Typ der Zielvariable ändert den Rechentyp des Ausdrucks nicht.

`long x = 50_000 * 50_000;`

`int`-Überlauf: −1.794.967.296

Keine automatische Fehlermeldung!

Dann trifft die Sonde den Mars nicht.



Zuweisung: Konstanten und Kurzformen

Code	Erlaubt?	Warum?
`byte b = 100;`	Ja	Der konstante Wert passt in `byte`.
`int n = 100;` `byte b = n;`	Nein	Eine `int`-Variable benötigt hier einen Cast.
`final int N = 100;` `byte b = N;`	Ja	`N` ist eine Konstante. Ihr Wert `100` passt in `byte` (−128 bis 127).
`byte b = 10;` `b += 1;`	Ja	Die Kurzzuweisung enthält eine Rückumwandlung.

`final` verhindert spätere Zuweisungen. Mit dem konstanten Anfangswert `100` kennt der Compiler den Wert schon beim Übersetzen und prüft: Er passt in `byte`. Ein eingelesener Wert reicht nicht.

Für diese Variable entspricht `b += 1;` der Form `b = (byte) (b + 1);`.
`b = b + 1;` ist dagegen ohne Cast unzulässig. Auch `b++` kann überlaufen.

Bitweise Operatoren

Ausdruck	Bitweise Rechnung	Wert

`6 & 3`	`  0110`
	`& 0011`
	`= 0010`
		`2`

`6 | 3`	`  0110`
	`| 0011`
	`= 0111`
		`7`

`6 ^ 3`	`  0110`
	`^ 0011`
	`= 0101`
		`5`

`~6`	Alle 32 Bits des `int` umkehren.	`-7`

Die Verknüpfung erfolgt für jede Bitposition einzeln.

Gezeigt sind die unteren 4 Bits.

Kurzzuweisungen:
`&=`, `|=`, `^=`

Auf Wahrheitswerten arbeiten `&`, `|` und `^` logisch.

Auf Ganzzahlen arbeiten sie bitweise.

Bits verschieben: <<, >>, >>>

Der linke Operand bestimmt den Ergebnistyp: `int` oder `long`. Kurzformen: `<<=`, `>>=`, `>>>=`.

Beispiel (`int`)	32 Bits: vorher / nachher	Wirkung
`6 << 1` `= 12`	`0`00000000 00000000 00000000 00000110 00000000 00000000 00000000 00001100	Links schieben, rechts mit 0 auffüllen.
`-8 >> 1` `= -4`	11111111 11111111 11111111 11111000 `1`1111111 11111111 11111111 11111100	Rechts schieben, links mit Vorzeichenbit auffüllen.
`-8 >>> 1` `= 2147483644`	11111111 11111111 11111111 11111000 `0`1111111 11111111 11111111 11111100	Rechts schieben, links mit 0 auffüllen.

Oben: vorher. Unten: nachher. **Rot: entfällt. Grün: neu aufgefüllt.**
Die Zahl rechts gibt die Anzahl der Stellen an: Bei `6 << 2` sind es `2` Stellen.

`>>` rundet nach unten: `-3 >> 1` ergibt `-2`, aber `-3 / 2` ergibt `-1`.

Anwendung: einzelne Berechtigungsbits

Unser Beispiel: Ein Benutzer darf eine Datei lesen und ausführen, aber nicht verändern.
Wir speichern jede Erlaubnis in einem Bit: `1` = erlaubt, `0` = nicht erlaubt.

Lesen `r = 4`	Schreiben `w = 2`	Ausführen `x = 1`
1	0	1

$101_2 = 5_8$: Lesen und Ausführen erlaubt.
In Java: `0b101` oder oktal `05`.

Unix: je eine Ziffer für Eigentümer, Gruppe und andere. `chmod 755 datei`: `rwx / r-x / r-x`.

`~lesen`: nur am Lesebit `0`, sonst `1`.
Unterste 3 Bits: `111 & 011 = 011`.
`& 0` löscht, `& 1` erhält ein Bit.

```
int rechte = 0b101;  // oktal: 05
int lesen = 0b100;
int schreiben = 0b010;

boolean darfLesen =
        (rechte & lesen) != 0;
rechte |= schreiben; // erlauben
rechte &= ~lesen;    // entziehen
rechte ^= schreiben; // umschalten
```

Die Zuweisungen wirken nacheinander:
`101` → `111` → `011` → `001`

Gleitkommazahlen

`float, double`



Kommazahlen: ein erster Ansatz mit fester Aufteilung

Festkommadarstellung: x Bits für den Vorkommateil, y Bits für den Nachkommateil.
Beispiel ohne Vorzeichen: x = 3 und y = 3.

3 Bits vor dem Komma **3 Bits nach dem Komma**

$2^2 = 4$	$2^1 = 2$	$2^0 = 1$	$2^{-1} = 0{,}5$	$2^{-2} = 0{,}25$	$2^{-3} = 0{,}125$
1	0	1	1	0	1

$$101{,}101_2 = 4 + 1 + 0{,}5 + 0{,}125 = 5{,}625_{10}$$

Das Komma ist fest vereinbart. Es belegt kein zusätzliches Bit.

3 Nachkommabits erlauben Schritte von $2^{-3} = 0{,}125$.

Bei fester Gesamtzahl der Bits begrenzt die Aufteilung Wertebereich und Genauigkeit.

Festkomma: Wertebereich und Genauigkeit

Beispiel: insgesamt 6 Bits, weiterhin ohne Vorzeichen.

Aufteilung der 6 Bits	Wertebereich	Schrittweite
3 vor, 3 nach dem Komma	0 bis 7,875	0,125
4 vor, 2 nach dem Komma	0 bis 15,75	0,25

$0{,}1_{10} = 0{,}0001100110011\ldots_2$

Die Bitfolge ist unendlich periodisch. Mit endlich vielen Nachkommabits bleibt nur eine Näherung.

Für mehr Wertebereich: das Komma mithilfe eines Exponenten verschieben. Auch dann bleibt 0,1 nur näherungsweise darstellbar.

Ein Java-Programm mit einem unerwarteten Ergebnis

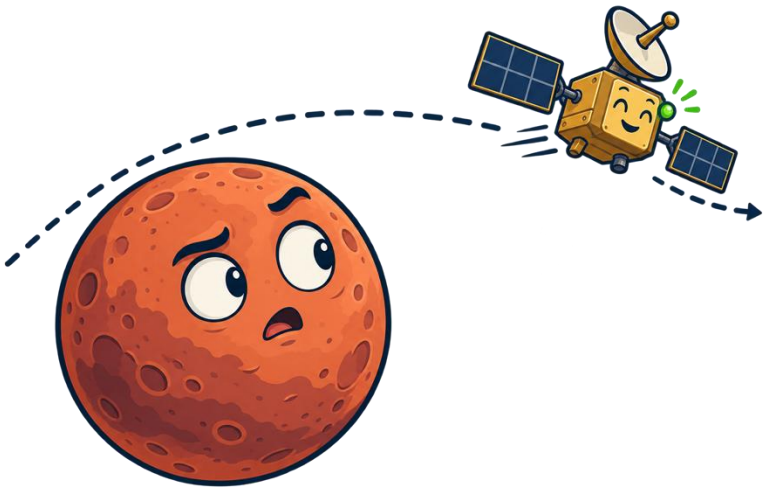
Mathematisch gilt: 0,3 − 0,2 = 0,1.

```
void main() {
    double ergebnis = 0.3 - 0.2;
    IO.println(ergebnis);
}
```

Ausgabe

```
0.09999999999999998
```

Auch 0,3 und 0,2 haben keine endliche Dualdarstellung.
Java rechnet hier mit bereits gerundeten Ausgangswerten.



Die Sonde trifft den Mars nicht.

Gleitkommadarstellung mit Exponent

Wissenschaftliche (halblogarithmische) Schreibweise

Dezimal: $823{,}625 = 8{,}23625 \cdot 10^2$
Dual: $1100110111{,}101_2 = 1{,}100110111101_2 \cdot 2^9$

Normalisierte Mantisse: vor dem Komma dezimal 1–9, dual immer 1.

Bestandteil	Im binären Beispiel	Aufgabe
Vorzeichen	positiv	Positiver oder negativer Wert
Mantisse	$1{,}100110111101_2$	Enthält die maßgeblichen Ziffern
Exponent	9	Verschiebt das Komma um neun Stellen

Java 25: Gleitkommatypen und IEEE 754

Die Mantisse normalisieren

Für Zahlen ungleich 0: Das Komma so verschieben, dass genau eine von 0 verschiedene Ziffer davor steht. Die passende Potenz gleicht die Verschiebung aus.

Basis	Komma nach links	Gleicher Wert mit normalisierter Mantisse
Dezimal	2 Stellen	$123{,}625 = 1{,}23625 \cdot 10^2$
Dual	6 Stellen	$1111011{,}101_2 = 1{,}111011101_2 \cdot 2^6$

Dezimal: 1 ≤ Mantisse < 10. Vor dem Komma steht eine Ziffer von 1 bis 9.
Dual: 1 ≤ Mantisse < 2. Vor dem Komma steht immer eine 1.

Die führende 1 ist bei normalisierten Dualzahlen bekannt und wird nicht mitkodiert.
Gespeichert wird nur der Nachkommateil der Mantisse, hier `111011101`.

Den Exponenten mit einem Zuschlag speichern

Exponenten können negativ sein. Ein fester Zuschlag (Bias) kodiert sie als nichtnegative Zahlen. Bei `float` verwenden wir 8 Bits und den Zuschlag 127.

Gespeicherter Wert = tatsächlicher Exponent + 127

Tatsächlicher Exponent	Gespeicherter Wert	8 Bits im Exponentfeld
−2	−2 + 127 = 125	`01111101`
0	0 + 127 = 127	`01111111`
6	6 + 127 = 133	`10000101`

IEEE 754: Aufbau von `float` und `double`

Datentyp	Vorzeichen	Exponent	Mantisse (gespeicherter Teil)	Gesamt
`float` (binary32)	1 Bit	8 Bits	23 Bits	32 Bits
`double` (binary64)	1 Bit	11 Bits	52 Bits	64 Bits

Bei normalisierten Zahlen speichern wir nur den Nachkommateil der Mantisse.
Die führende 1 ergibt sich aus der Normalisierung. Das liefert 24 bzw. 53 Bits Präzision.

Zuschlag zum Exponenten (Bias): 127 bei `float`, 1023 bei `double`.
Nur Nullen bzw. nur Einsen im Exponentfeld sind Kennungen für besondere Werte.

Beispiel: 123,625 als `float` kodieren

$123{,}625_{10} = 1111011{,}101_2 = 1{,}111011101_2 \cdot 2^6$

Feld	Herleitung	Gespeicherte Bits
Vorzeichen	positiv	`0`
Exponent	6 + 127 = 133	`10000101`
Mantisse	Führende 1 weglassen, mit Nullen auffüllen.	`11101110100000000000000`

```
0 10000101 11101110100000000000000
```

Normalisierte IEEE-754-Werte: Vorzeichen, Mantisse und Exponent

Von 32 Bits zurück zur Kommazahl

```
0 10000101 11101110100000000000000
```

Feld	Gespeicherte Bits	Interpretation
Vorzeichen	`0`	Positiver Wert
Exponent	`10000101`	$10000101_2 = 133$; $133 - 127 = 6$
Mantisse	`11101110100000000000000`	Führende 1 ergänzen: $1{,}111011101_2$

$$1{,}111011101_2 \cdot 2^6 = 1111011{,}101_2$$

$$64 + 32 + 16 + 8 + 2 + 1 + 0{,}5 + 0{,}125 = 123{,}625$$

Zum Ausprobieren: IEEE-754-Konverter für `float` (32 Bit)

`float`: 32 Bits für Gleitkommazahlen

Etwa 6 bis 7 signifikante Dezimalstellen

```
float temperatur = 21.5f;
float klein = 0.001f;
// float wert = 0.1; // Compilerfehler
```

Größe	Ungefährer Wert
Größter endlicher Betrag	$3{,}4028235 \cdot 10^{38}$
Kleinster positiver Wert (subnormal)	$1{,}4 \cdot 10^{-45}$

`f` oder `F` kennzeichnet ein `float`-Literal. Die Genauigkeit betrifft die gesamte Zahl, nicht eine feste Anzahl von Nachkommastellen.

`double`: 64 Bits und höhere Genauigkeit

Etwa 15 bis 16 signifikante Dezimalstellen

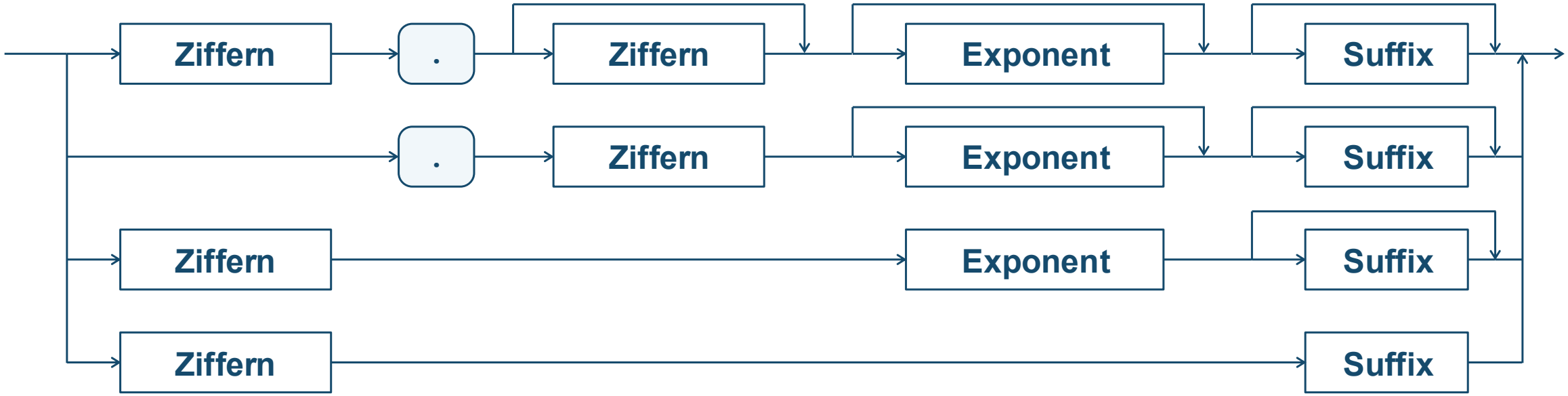
```
double temperatur = 21.5;
double klein = 0.001;
double ebenfalls = 21.5d;
```

Größe	Ungefährer Wert
Größter endlicher Betrag	$1{,}7976931348623157 \cdot 10^{308}$
Kleinster positiver Wert (subnormal)	$4{,}9 \cdot 10^{-324}$

Ohne Suffix hat ein Gleitkommaliteral den Typ `double`. `d` oder `D` ist optional.
Für allgemeine Gleitkommarechnungen verwenden wir meist `double`.

Gleitkommaliterale in Java

<Gleitkommaliteral> (dezimale Schreibweisen)



Ziffern: eine oder mehrere Dezimalziffern. Exponent: `e`/`E`, optional `+`/`-`, dann Ziffern.
Suffix: `f`, `F`, `d` oder `D`. Beispiele: `.5`, `5.`, `2f`, `1e3`, `1.25e-3f`.

`1.25e-3` bedeutet $1{,}25 \cdot 10^{-3} = 0{,}00125$.

Unterstriche und hexadezimale Gleitkommaliterale: JLS §3.10.2

Rechnen mit Gleitkommazahlen

Operatoren	Beispiel	Ergebnis
`+`, `-`, `*`, `/`	`5.0 / 2.0`	`2.5`
Restoperator `%`	`5.5 % 2.0`	`1.5`
Unäres `+` und `-`	`-2.5`	`-2.5`
`++`, `--`	`double x = 2.5; x++;`	Danach: `x` ist `3.5`
`<`, `<=`, `>`, `>=`, `==`, `!=`	`2.5 < 3.0`	`true`

Kurzzuweisungen: `+=`, `-=`, `*=`, `/=`, `%=`.
Bitweise Operatoren und Schiebeoperatoren sind für Gleitkommazahlen unzulässig.

Multiplikation: Rundung bei `float`

```
float a = 4096.5f, b = 4096.5f;
float produkt = a * b; // gespeichert: 16781312
```

Exakt: 4096,5 · 4096,5 = 16 781 312,25

24 maßgebliche Bits (blau), übrige Bits (rot)

`1,0000 0000 0001 0000 0000 000` | `001` $_2 \cdot 2^{24}$

Komma um 24 Stellen nach rechts:

`1000 0000 0000 1000 0000 0000` | `0,01` $_2$

Rot: 0 · 1 + 0 · 0,5 + 1 · 0,25 = 0,25. Der Computer verrechnet sich um 0,25.

Beim Runden bleiben die behaltenen Bits unverändert, oder dieser Bitteil wird um 1 erhöht. Java wählt die Möglichkeit, die näher am exakten Ergebnis liegt.

Beim Erhöhen kann ein Übertrag entstehen. +1 im Bitteil erhöht hier den Zahlenwert um 2.

16 781 312 (Abstand: 0,25): gewählt 16 781 314 (Abstand: 1,75)

Java 25: Multiplikation und Rundung

Multiplikation: Aufrunden mit Übertrag

```
float a = 4097.5f, b = 4098.5f;
float produkt = a * b; // gespeichert: 16793604
```

Exakt: 4097,5 · 4098,5 = 16 793 603,75

24 maßgebliche Bits (blau), übrige Bits (rot)

`1,0000 0000 0100 0000 0000 001` | $111_2 \cdot 2^{24}$

Komma um 24 Stellen nach rechts:

`1000 0000 0010 0000 0000 0001` | $1{,}11_2$

Rot: 1 · 1 + 1 · 0,5 + 1 · 0,25 = 1,75.

16 793 602 (Abstand: 1,75) **16 793 604 (Abstand: 0,25): gewählt**

Aufrunden mit Übertrag: Der behaltene Bitteil endet auf 1 und wird um 1 erhöht.

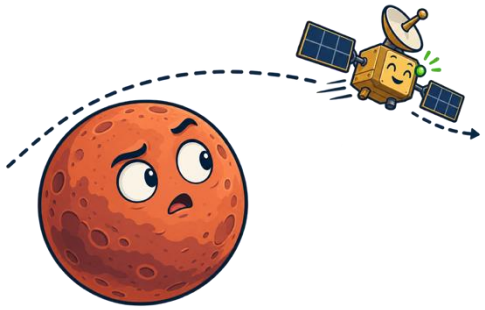
`…0001 + 1 = …0010` Hier: Zahlenwert +2.

Ergebnis um 0,25 zu groß: 2 − 1,75 = 0,25.

Java 25: Multiplikation und Rundung

Rundungsfehler verstehen

```
double summe = 0.1 + 0.2;
IO.println(summe);          // 0.30000000000000004
IO.println(summe == 0.3); // false
```



Für den Vergleich bilden wir den Betrag der Differenz zum erwarteten Wert:

```
double eps = 1e-6; // Epsilon (Toleranz): 0.000001
double differenz = summe - 0.3;
if (differenz < 0) {
    differenz = -differenz;
}
boolean nahe = differenz <= eps; // true
```

`eps` muss zur Aufgabe und Größenordnung passen. Es gibt keinen universellen Wert.

Wo möglich, ganzzahlig rechnen: `long preisInCent = 1234; // 12,34 Euro`

Besondere Werte und Grenzen

Ausdruck	Ergebnis	Bedeutung
`1.0 / 0.0`	`Infinity`	Positive Unendlichkeit
`-1.0 / 0.0`	`-Infinity`	Negative Unendlichkeit
`0.0 / 0.0`	`NaN`	Not a Number: kein Zahlenwert
`1.0e308 * 10`	`Infinity`	Überlauf
`1.0e-300 / 1.0e100`	`0.0`	Unterlauf bis zur null

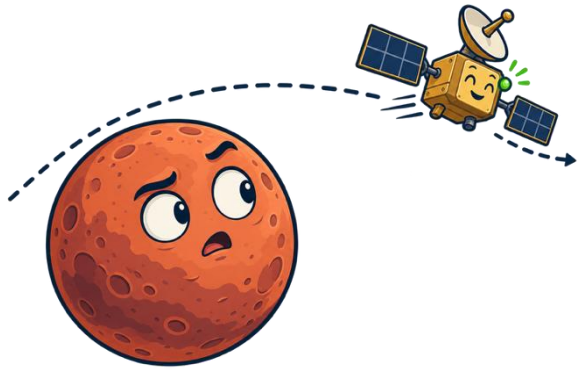
Bei `NaN` sind `<`, `<=`, `>`, `>=` und `==` immer `false`. `!=` ist `true`.
Es gibt positive und negative null. Sie sind mit `==` gleich.

IEEE 754 in Java: NaN, Unendlichkeiten und vorzeichenbehaftete null

Implizit erweitern: größer heißt nicht immer genauer

Umwandlung	Automatisch erlaubt?	Wert bleibt exakt?
`int` nach `double`	Ja	Ja
`float` nach `double`	Ja	Ja, für den bereits gespeicherten Wert
`int` nach `float`	Ja	Nicht immer
`long` nach `float` oder `double`	Ja	Nicht immer

```
int ganz = 16_777_217;
float naeherung = ganz;
IO.println((int) naeherung); // 16777216
```



Schon Zeile 2 rundet auf 16.777.216.
Keine automatische Fehlermeldung!

Erweiternde Umwandlungen können Präzision verlieren

Explizit umwandeln: Gleitkommazahl und Ganzzahl

```
double wert = 3.9;
int ganz = (int) wert;  // 3
int negativ = (int) -3.9; // -3
float kleiner = (float) wert;
```

Beim Cast zur Ganzzahl entfällt der Nachkommaanteil in Richtung null.
Das ist kein Runden zur nächstgelegenen ganzen Zahl.
Beim Cast von `double` nach `float` kann zusätzlich Genauigkeit verloren gehen.

Auch ein Cast kann Überlauf erzeugen: `(float) 1e100` ergibt `Infinity`.
Die ursprüngliche Variable `wert` bleibt durch den Cast unverändert.

Der Rechentyp bei gemischten Operanden

Für die üblichen binären Rechenoperatoren gilt diese Reihenfolge:



Der weiter rechts stehende beteiligte Typ bestimmt den Rechentyp.
`byte` und `short` werden zuvor mindestens zu `int` erweitert.

Ausdruck	Rechentyp	Wert
`5 / 2`	`int`	`2`
`5 / 2L`	`long`	`2`
`5 / 2.0f`	`float`	`2.5`
`5 / 2.0`	`double`	`2.5`

Der Zeitpunkt der Umwandlung entscheidet

Code	Wert	Was passiert zuerst?
`double a = 5 / 2;`	`2.0`	Ganzzahldivision, danach Erweiterung
`double b = (double) (5 / 2);`	`2.0`	Ganzzahldivision, danach Cast
`double c = (double) 5 / 2;`	`2.5`	Cast eines Operanden, danach Division
`long d = 50_000 * 50_000;`	`-1794967296`	Überlauf der int-Multiplikation
`long e = (long) 50_000 * 50_000;`	`2500000000`	Cast vor der Multiplikation

Eine größere Zielvariable kann ein zuvor falsch berechnetes oder gerundetes Ergebnis nicht nachträglich reparieren.

Wichtige Rechenoperationen mit `Math`

```
double wurzel = Math.sqrt(9); // Ergebnis: 3.0
```

Aufruf	Ergebnis	Bedeutung
`Math.abs(-7)`	`7`	Betrag: Abstand von null
`Math.min(4, 9)` `Math.max(4, 9)`	`4` `9`	Kleineren bzw. größeren Wert bestimmen
`Math.sqrt(9)`	`3.0`	Quadratwurzel
`Math.pow(2, 3)`	`8.0`	Potenz: 2^3
`Math.round(2.6)`	`3`	Zur nächstgelegenen ganzen Zahl runden
`Math.floor(2.8)`	`2.0`	Nach unten runden
`Math.ceil(2.2)`	`3.0`	Nach oben runden
`Math.random()`	`z. B. 0.42`	Zufallszahl von 0.0 (inklusive) bis 1.0 (exklusive)

`round(double)` liefert `long`, `round(float)` liefert `int`. `floor` und `ceil` liefern `double`.

Weitere Methoden und Details: Java 25 – Math-API

Zeichen

`char`



Zeichen kodieren: die Idee von ASCII

Eine Zeichentabelle ordnet jedem enthaltenen Zeichen eine Zahl zu.
ASCII verwendet 7 Bits und umfasst die Codes 0 bis 127.

Zeichen	Dezimal	Hexadezimal	Binär (mit führender 0)
`'A'`	65	41	`01000001`
`'a'`	97	61	`01100001`
`'0'`	48	30	`00110000`
`' '` (Leerzeichen)	32	20	`00100000`

Ein Zeichencode ist eine Zahl. Das Schriftbild erzeugt erst die verwendete Schriftart.

Offizielle Zeichentabelle: Unicode Basic Latin / ASCII

Die ASCII-Tabelle

Hex	0x0…	0x1…	0x2…	0x3…	0x4…	0x5…	0x6…	0x7…
0	NUL	DLE	SP	0	@	P	`	p
1	SOH	DC1	!	1	A	Q	a	q
2	STX	DC2	"	2	B	R	b	r
3	ETX	DC3	#	3	C	S	c	s
4	EOT	DC4	$	4	D	T	d	t
5	ENQ	NAK	%	5	E	U	e	u
6	ACK	SYN	&	6	F	V	f	v
7	BEL	ETB	'	7	G	W	g	w
8	BS	CAN	(	8	H	X	h	x
9	HT	EM	)	9	I	Y	i	y
A	LF	SUB	*	:	J	Z	j	z
B	VT	ESC	+	;	K	[	k	{
C	FF	FS	,	<	L	\	l	|
D	CR	GS	-	=	M	]	m	}
E	SO	RS	.	>	N	^	n	~
F	SI	US	/	?	O	_	o	DEL

Lesen: Spalte 0x4… und Zeile 1 ergeben 0x41 = 65 = A.
SP: Leerzeichen. Codes 0 bis 31 und 127 sind Steuerzeichen.

`char`: ein Zeichenwert in Java

16 Bits ohne Vorzeichen: 0 bis 65.535

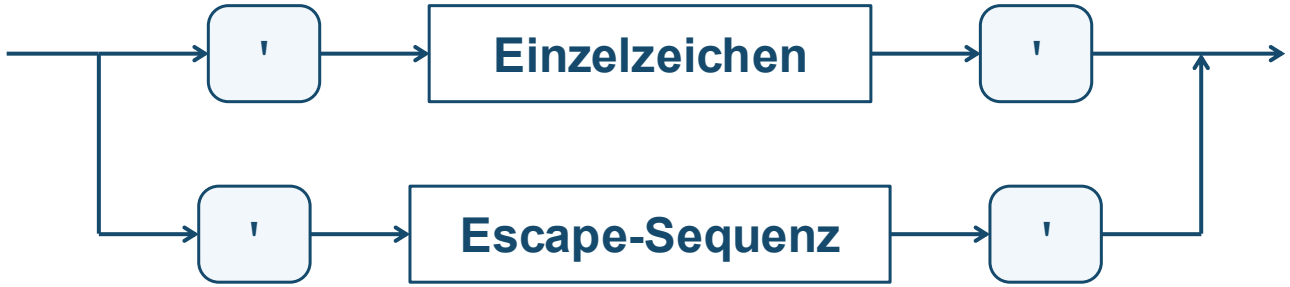
```
char buchstabe = 'A';
char ziffer = '7';
char leerzeichen = ' ';
char umlaut = '\u00E4'; // ä
```

Ein `char` enthält eine UTF-16-Codeeinheit.
Die ASCII-Codes behalten ihre Bedeutung, weitere Codes ergänzen andere Zeichen.
Zeichenliterale verwenden einfache Anführungszeichen.

`'7'` ist ein Zeichen mit Code 55. `7` ist eine Zahl. `"7"` ist eine Zeichenkette.

Zeichenliterale und Escape-Sequenzen

<Zeichenliteral>



Einzelzeichen: ohne einfaches Anführungszeichen, Rückstrich oder Zeilenumbruch.

Escape	Bedeutung	Escape	Bedeutung
`\n`	Zeilenumbruch	`\t`	Tabulator
`\r`	Wagenrücklauf	`\b`	Rückschritt
`\'`	Einfaches Anführungszeichen	`\\`	Rückstrich

Weitere Escape-Sequenzen: Java-Sprachspezifikation §3.10.7

Unicode und die Grenzen eines einzelnen `char`

Unicode ordnet Zeichen Nummern (Codepunkte) zu. UTF-16 kodiert diese Nummern.

Zeichen	Unicode-Codepunkt	UTF-16-Codeeinheiten
A	U+0041	`0041`
ä	U+00E4	`00E4`
😀	U+1F600	`D83D DE00`

Ein `char` speichert genau eine 16-Bit-Codeeinheit.
Für 😀 braucht UTF-16 zwei: `D83D` und `DE00`.
Erst beide zusammen kodieren das Emoji. Ein einzelner `char` reicht nicht.

Ein sichtbares Zeichen kann auch aus mehreren Codepunkten bestehen.

Unicode: Codepunkte, UTF-16 und Surrogatpaare

Mit Zeichencodes rechnen und vergleichen

```
char zeichen = 'A';
int code = zeichen;              // 65, implizit
char naechstes = (char) (zeichen + 1); // 'B'
int wert = '7' - '0';            // 7
boolean gross = 'A' <= zeichen && zeichen <= 'Z';
```

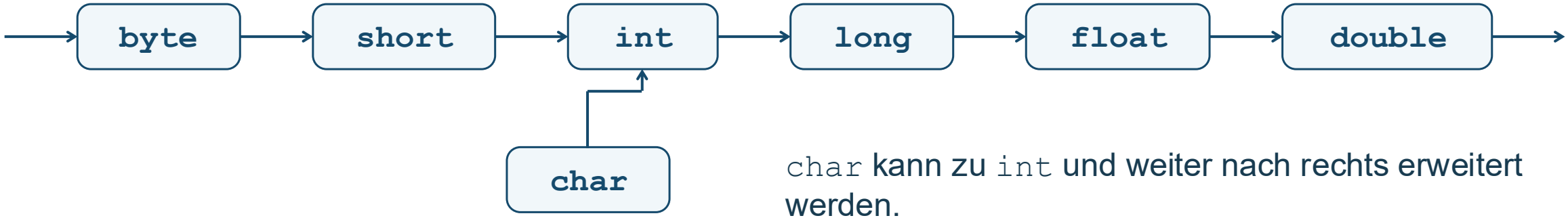
`char` gehört in Java zu den ganzzahligen Typen. Beim Rechnen entsteht meist `int`.
Die numerischen Operatoren sind erlaubt. `zeichen++` erhöht den Code um 1.

Die Bereichsprüfung oben erkennt nur A bis Z. Sie erfasst beispielsweise kein Ä.
`'A' + 'B'` ergibt die Zahl 131, keine Zeichenkette.

Welche Datentypen sind „stärker“?

Für gewöhnliche binäre Zahlenoperationen gilt: `double` vor `float` vor `long` vor `int`.
`byte`, `short` und `char` werden mindestens zu `int` erweitert.

Automatisch erlaubte Erweiterungen bei einer Zuweisung



`char` kann zu `int` und weiter nach rechts erweitert werden.

`short` und `char` sind nicht ineinander einzuordnen. `boolean` steht außerhalb.
Die Pfeile bedeuten „automatisch erlaubt“, nicht „garantiert ohne Genauigkeitsverlust“.

Java 25: Typumwandlungen und numerische Promotion

Zeichenketten

`String`

Kein primitiver Datentyp



Zeichenketten mit `String`

Ein Text kann aus keinem, einem oder vielen Zeichen bestehen.

```
String name = "Ada";
String leer = "";
String ziffer = "7";
String zeile = IO.readln("Name: ");
```

`String` ist ein Referenztyp und kein primitiver Datentyp.
Die Variable `name` enthält einen Verweis (eine Referenz) auf den Text `"Ada"`.
Der Text liegt an anderer Stelle im Speicher. Die Referenz kann man sich vereinfacht wie eine Speicheradresse vorstellen.

Java 25: String

`String`-Literale und Textblöcke

<Stringliteral>



Zwischen den Anführungszeichen: null oder mehr Zeichen beziehungsweise Escapes.
`"` und `\` benötigen einen Escape. Ein direkter Zeilenumbruch ist hier unzulässig.

```
String text = "Er sagt: \"Hallo!\"\nNeue Zeile.";
```

Mehrere Zeilen als Textblock

Nach dem öffnenden `"""` folgt ein Zeilenumbruch. Java entfernt gemeinsame beiläufige Einrückung.

```
String text = """
        Erste Zeile
        Zweite Zeile
        """;
```

String-Literale und Textblöcke: JLS §3.10.5 und §3.10.6

Texte verbinden mit + und +=

Ausdruck	Ergebnis
`"Hallo, " + "Ada"`	`"Hallo, Ada"`
`"Summe: " + 2 + 3`	`"Summe: 23"`
`"Summe: " + (2 + 3)`	`"Summe: 5`
`'A' + 'B'`	`131`
`"" + 'A' + 'B'`	`"AB"`

`+` ist linksassoziativ. Sobald ein Operand ein `String` ist, verbindet `+` Text.
`String text = "Hallo"; text += "!";` ergibt anschließend `"Hallo!"`.

Textinhalte mit `.equals()` vergleichen

```
String antwort = IO.readln("Gib ja ein: ");

// Bei Eingabe ja:
IO.println(antwort == "ja");       // false: anderer Verweis
IO.println(antwort.equals("ja")); // true: gleicher Inhalt
```

Zur Erinnerung: `String`-Variablen enthalten Referenzen (Verweise).
`==` vergleicht die Verweise: Führen beide zur selben Stelle im Speicher?
Gleicher Textinhalt kann auch an verschiedenen Speicherstellen liegen.
`.equals()` prüft den Textinhalt. Groß- und Kleinschreibung zählen mit.

`String` unterstützt keine Inhaltsvergleiche mit <, <=, > oder >=.

Java 25: String.equals()

Vorgegebene Operationen für Texte

Für `String text = "Java";` gilt:
Auch direkt möglich: `"Hallo".length()` liefert `5`, ohne vorherige Variable.

Operation	Ergebnis	Bedeutung
`text.length()`	`4`	Anzahl der UTF-16-Codeeinheiten
`text.charAt(0)`	`'J'`	Codeeinheit an Position 0
`text.substring(1, 3)`	`"av"`	Ab Position 1 bis vor Position 3
`text.equals("Java")`	`true`	Gleicher Textinhalt
`text.isEmpty()`	`false`	Prüfung auf die leere Zeichenkette

Positionen beginnen bei 0. Bei `"Java"` sind für `charAt()` die Positionen 0 bis 3 erlaubt.
Eine Position außerhalb des Bereichs führt zu einem Laufzeitfehler.

Java 25: String-Operationen

`String`-Werte sind unveränderlich

```
String a = "Hallo";
String b = a;
a += "!";

IO.println(a); // Hallo!
IO.println(b); // Hallo
```

Die Verkettung liefert einen neuen `String`-Wert.

`a` erhält diesen neuen Wert.
`b` behält den bisherigen Wert.

Die Variable darf also ihren Wert wechseln, der Textwert selbst ändert sich nicht.

Auch `substring()` liefert einen Ergebniswert. Der bisherige Text bleibt erhalten.

UTF-16 im Programm und UTF-8 in Dateien

Text	UTF-16-Codeeinheiten	UTF-8-Bytes
"A"	0041	41
"ä"	00E4	C3 A4
"😀"	D83D DE00	F0 9F 98 80

```
String text = "😀";
IO.println(text);          // ein sichtbares Emoji
IO.println(text.length()); // 2
```

`length()` zählt Codeeinheiten: 2 `char`-Werte, obwohl wir nur 1 Emoji sehen.
Die UTF-8-Kodierung desselben Emojis benötigt dagegen 4 Bytes.
Unicode: Vergleich von UTF-8 und UTF-16

Text und Zahlen ineinander umwandeln

```
String eingabe = IO.readln("Messwert: ");
double wert = Double.parseDouble(eingabe);
IO.println("Doppelter Wert: " + (wert * 2));
```

Umwandlung	Werkzeug
Text in Ganzzahl	`Integer.parseInt(text)`, `Long.parseLong(text)`
Text in Gleitkommazahl	`Float.parseFloat(text)`, `Double.parseDouble(text)`
Wert in Text	`String.valueOf(wert)` oder Verkettung mit einem `String`

Ein Cast wie `(int) "42"` ist unzulässig. Parsen wertet die Zeichen als Zahl aus.
Hier wird der Dezimalpunkt erwartet. Ungültiger Zahlentext führt zu einem Laufzeitfehler.

Java-Operatoren nach Priorität

Je höher die Priorität, desto stärker die Bindung. Klammern legen die Gruppierung fest.

Priorität	Gruppe	Operatoren	Gruppierung
15	Postfix	`x++  x--`	–
14	Unär und Cast	`++x  --x  +x  -x  !x  ~x  (Typ) x`	rechts
13	Multiplikation, Division, Rest	`*  /  %`	links
12	Addition, Subtraktion, Text	`+  -`	links
11	Bitverschiebung	`<<  >>  >>>`	links
10	Vergleich / Typprüfung*	`<  <=  >  >=  instanceof`	links
9	Gleichheit / Ungleichheit	`==  !=`	links
8	Bitweises / logisches UND	`&`	links
7	Bitweises / logisches XOR	`^`	links
6	Bitweises / logisches ODER	`|`	links
5	UND mit Kurzschluss	`&&`	links
4	ODER mit Kurzschluss	`||`	links
3	Bedingter Ausdruck	`? :`	rechts
2	Zuweisung und Kurzformen	`= += -= *= /= %= &= ^= |= <<= >>= >>>=`	rechts
1	Lambda-Ausdruck*	`->`	rechts

Gruppierung bei gleichem Rang: links z. B. `(a - b) - c`, rechts z. B. `a = (b = c)`.

* `instanceof` und Lambda-Ausdrücke folgen später. Nachschlagen: Java 25, JLS Kapitel 15.

Welcher Datentyp passt zur Aufgabe?

Aufgabe	Übliche Wahl	Worauf achten?
Bedingung oder Zustand	`boolean`	Zwei Wahrheitswerte
Zähler oder gewöhnliche ganze Zahl	`int`	Wertebereich und Überlauf
Große ganze Zahl	`long`	`L` bei großen Literalen und vor Rechnungen
Begrenzte Ganzzahl mit passender Breite	`byte`, `short`	Vorzeichen und Rechnen als `int`
Allgemeine Gleitkommarechnung	`double`	Rundung und passende Genauigkeit
Bewusst reduzierte Präzision	`float`	`f`-Suffix und begrenzte Genauigkeit
Einzelne UTF-16-Codeeinheit / Text	`char` / `String`	Unicode und Inhaltsvergleich

Der passende Typ richtet sich nach den Werten und benötigten Operationen.

Dokumentation zum Nachschlagen

Datentypen, Wertebereiche und Kodierung

Literale, Zeichen und Escape-Sequenzen

Implizite / explizite Umwandlungen und numerische Promotion

Operatoren und Auswertung von Ausdrücken

String: verfügbare Operationen

ASCII-Tabelle und Unicode-Kodierungen

Nur Nullen und Einsen?



Lernziele für die anschließende Übung

Das trainieren Sie mit kleinen Java-Programmen:

Werte darstellen

Zwischen Zahlensystemen umrechnen und Bitmuster erklären.

Operatoren gezielt einsetzen

Logische, arithmetische und bitweise Ausdrücke auswerten.

Typumwandlungen prüfen

Rechentypen bestimmen, Casts anwenden und Informationsverluste erkennen.

Außerdem: Zeichen und Texte verarbeiten sowie Grenzfälle gezielt testen.